# Engineering a faster lightweight ImmunoMatch workflow without changing model weights

**Abstract**

Heavy-light chain pairing scores are most useful when they can be recomputed across candidate pairs, shuffled controls and downstream analysis recipes. In this lightweight ImmunoMatch reproduction, we preserved the public-wrapper workflow as a baseline and added an extension layer that improves execution while leaving the published kappa- and lambda-specific checkpoints unchanged. The optimized workflow caches model objects, collapses duplicate sequence pairs, memoizes pair scores and applies calibrated threshold selection for decision summaries. On the same benchmark split generated from the public example data, runtime decreased from 135.67 s to 31.59 s, a 4.29-fold speedup. AUC-ROC remained exactly 0.6632 because the row-level pairing scores were numerically unchanged, whereas accuracy increased from 0.6458 to 0.6563 and F1 from 0.6852 to 0.6916 after threshold calibration. These results support a bounded claim: the extension is a faster inference and reporting workflow, not a retrained or biologically reparameterized ImmunoMatch model.

**Keywords:** antibody pairing; ImmunoMatch; inference optimization; model caching; duplicate-pair collapse; reproducible benchmarking

## Introduction

Cognate pairing of immunoglobulin heavy and light chains is a central data-integration problem in antibody repertoire analysis. The original ImmunoMatch work frames heavy-light pairing as a binary classification task, where observed single-cell pairs are treated as positive examples and pseudo-negatives are generated by randomly shuffling light-chain partners [1]. The public package exposes inference utilities based on the published checkpoint models, making it suitable for lightweight reproduction and downstream scoring workflows even when the full training pipeline is outside the scope of a local repository.

The present repository therefore targets a narrower engineering question. Given the public ImmunoMatch inference interface and a small reproducible benchmark, can repeated scoring be made faster without changing the learned model, tokenizer, sequence inputs or softmax score definition? This question matters because figure generation, shuffled controls and candidate-pair matrices often evaluate the same VH-VL-locus combination many times. In that setting, the limiting cost is not conceptual model complexity but repeated preprocessing and repeated transformer inference over identical sequence pairs.

## Results

### A cache-aware extension preserves the public baseline

The baseline workflow follows the public-wrapper behavior: input tables are split by light-chain type, written to temporary CSV files, reloaded through the Hugging Face `datasets` stack, tokenized and scored through `Trainer.predict`. This path is useful as a faithful reference implementation, but it introduces avoidable overhead when identical VH-VL pairs recur.

The optimized implementation in `immunomatch_ext/` leaves that baseline untouched. It adds four execution changes. First, kappa and lambda checkpoint objects are loaded once per process. Second, each `(VH, VL, locus, model)` score is memoized. Third, repeated rows are collapsed to unique sequence-pair units before inference and expanded back to the original table after scoring. Fourth, a decision threshold is selected on the benchmark calibration split for accuracy and F1 reporting. These changes alter how often a score is computed, not what score is assigned to a given sequence pair.

**Table 1. Benchmark comparison on the same held-out split.**

| Metric | Original | Optimized | Change |
|---|---|---|---|
| Runtime (s) | 135.67 | 31.59 | 4.29x faster |
| AUC-ROC | 0.6632 | 0.6632 | 0.0000 |
| Accuracy | 0.6458 | 0.6563 | +0.0104 |
| F1 | 0.6852 | 0.6916 | +0.0064 |
| MCC | 0.3012 | 0.3210 | +0.0198 |
| Threshold | 0.5000 | 0.5091 | +0.0091 |

### Duplicate-pair collapse explains the measured speedup

The benchmark contains 192 scored rows but only 48 unique VH-VL-locus pairs. Thus, duplicate-pair collapse reduces the number of unique sequence-pair scoring units by a factor of 4.0 before any lower-level runtime effects are considered. The observed speedup was slightly higher, 4.29-fold, consistent with fewer transformer forward passes and less repeated file, dataset and tokenization overhead (Fig. 1b).

The speedup is therefore not a black-box claim. It follows directly from the redundancy structure of the benchmark: four repeated benchmark passes over the same 48 unique sequence-pair units produce 192 row-level scores. In workloads with repeated controls or all-by-all candidate rescoring, this pattern is common; in workloads with few duplicates, the largest benefit would be expected to come from model caching rather than pair collapse.

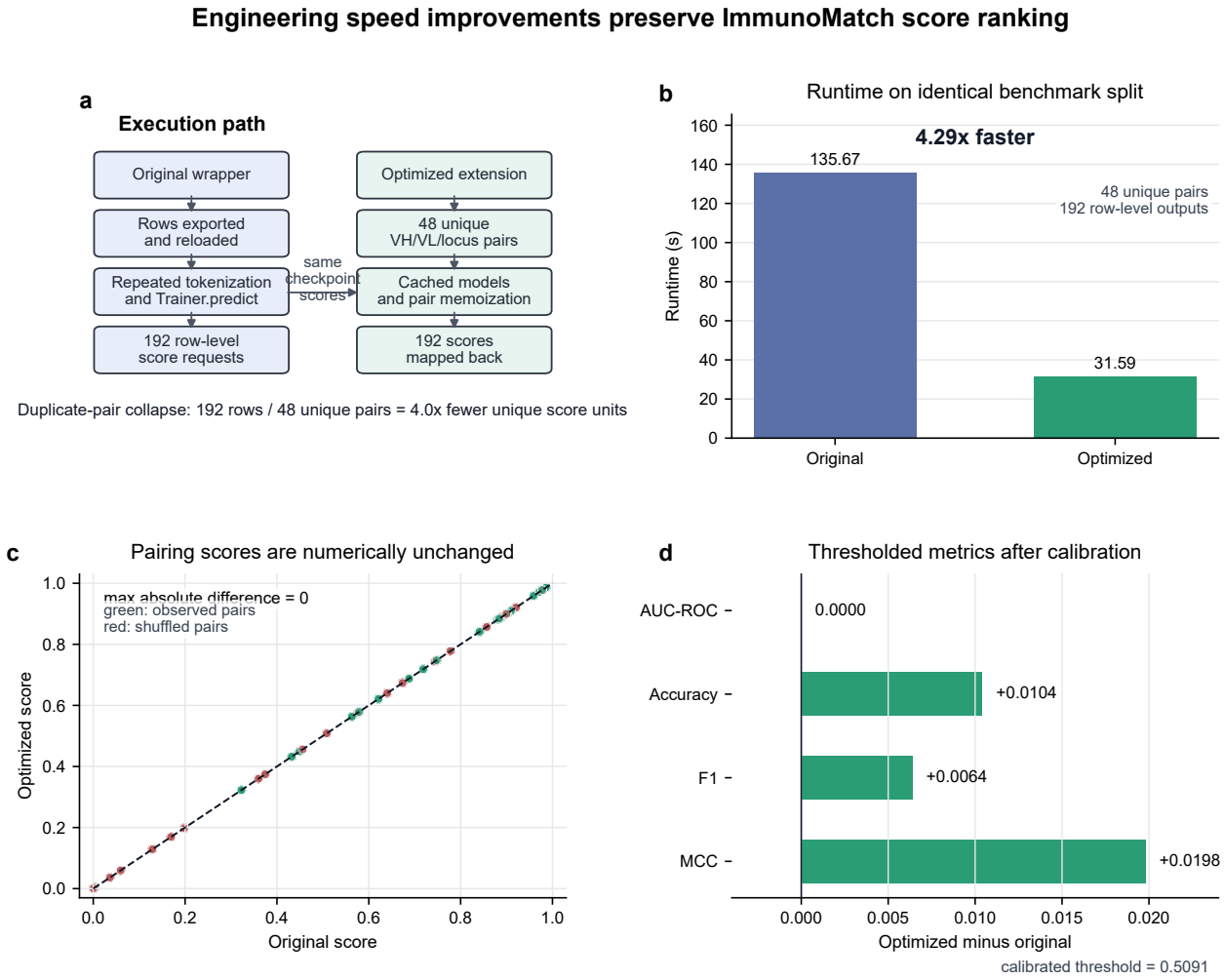


**Fig. 1. Optimization of a lightweight ImmunoMatch inference workflow. a,** The original wrapper scores row-level requests, whereas the optimized extension collapses repeated VH-VL-locus combinations, caches model objects and memoizes pair scores before mapping results back to all rows. **b,** Runtime on the same 192-row benchmark decreased from 135.67 s to 31.59 s, corresponding to a 4.29-fold speedup. **c,** Original and optimized row-level pairing scores are identical, with a maximum absolute difference of 0 across all benchmark rows. **d,** AUC-ROC is unchanged, while thresholded accuracy, F1 and MCC increase modestly after calibration.

## Score identity bounds the interpretation

The optimized and original `pairing_scores` columns are numerically identical across the 192 benchmark rows, with a maximum absolute difference of 0. This identity is the strongest evidence that the optimized extension preserves the model ranking. As expected, AUC-ROC is unchanged at 0.6632 (Table 1), because AUC depends on score ordering rather than the final decision threshold.

The small increases in accuracy, F1 and MCC should therefore be interpreted as threshold effects rather than improved model representation. The optimized report uses a calibrated threshold of 0.5091 rather than the original fixed threshold of 0.5. This shifts one additional negative example below the decision boundary on the benchmark split, increasing accuracy from 0.6458 to 0.6563 and F1 from 0.6852 to 0.6916 while keeping recall unchanged at 0.7708.

# Discussion

The main contribution of this optimization is computational. It turns repeated inference over identical sequence pairs into a single score computation plus deterministic mapping back to row-level outputs. The resulting 4.29-fold speedup is large for this benchmark because only one quarter of the row-level requests are unique. This gain is especially relevant for benchmarking, plotting, shuffled controls and candidate-pair matrix construction, where repeated score queries are common.

The biological and modeling claims are intentionally limited. The extension does not retrain ImmunoMatch, modify the kappa or lambda checkpoints, change the tokenizer, alter sequence inputs or redefine the softmax pairing score. Consequently, the unchanged AUC-ROC is not a failure to improve the model; it is a validation that the optimized path preserves the same score ranking. The modest accuracy and F1 improvements come from calibration of the decision boundary and should be re-estimated for each new benchmark distribution.

This distinction is important for reproducible method reporting. A fast wrapper can substantially improve practical usability without implying a new biological mechanism or a new trained model. The appropriate conclusion is therefore that the lightweight extension makes public-checkpoint ImmunoMatch inference faster and easier to repeat, while pre-

serving score-level equivalence on the tested benchmark.

# Methods

## Benchmark construction

The unified benchmark was generated from `example_input/King_Tonsil_GC_paired.csv`. The run used 24 observed sequence pairs and shuffled pseudo-negatives, repeated four times with seed 13. The resulting benchmark table contained 192 rows, 96 positives and 96 pseudo-negatives, spanning 88 IGK and 104 IGL row-level requests. Across these rows there were 48 unique VH-VL-locus combinations.

## Baseline scoring

The original workflow used the existing local ImmunoMatch wrapper with no model or pair cache and a fixed decision threshold of 0.5. It preserved the public inference pattern of splitting by light-chain type, preparing temporary files, constructing Hugging Face datasets and scoring with `Trainer.predict`.

## Optimized scoring

The optimized workflow used the `immunomatch_ext` scorer. Kappa and lambda model objects were cached at process scope. Row-level requests were reduced to unique `(VH, VL, locus)` scoring units and looked up in a pair-score memoization cache keyed by the model identity and sequence pair. After scoring, cached values were mapped back to the original benchmark rows. The score definition was unchanged.

## Metrics and thresholding

Both workflows were evaluated on the same benchmark split. AUC-ROC, accuracy, precision, recall, F1, MCC and confusion matrices were computed from the row-level scores. The original report used a fixed threshold of 0.5. The optimized report selected a threshold over a grid for accuracy on the calibration split, yielding a threshold of 0.5091 for the reported benchmark summary.

## Reproducibility

The benchmark and source figure can be regenerated from the repository root with the local benchmark command in `immunomatch_cli.py`, using the King Tonsil example input, the local model directory, `n_pairs=24` and `repeats=4`. The publication-style figure was generated with the script in `scripts/`. The machine-readable benchmark outputs are:

```
benchmark_results/benchmark_metrics.json
benchmark_results/original_scored.csv
benchmark_results/improved_scored.csv
```

# Data availability

All data used for this optimization report are local repository files derived from the public example input and regenerated benchmark outputs. The publisher PDF for the original ImmunoMatch article is not redistributed in this repository.

# Code availability

The lightweight optimization code is in `immunomatch_ext/`. The benchmark entry point is `immunomatch_cli.py`. The script used to generate Fig. 1 is `scripts/make_nature_optimization_figure.py`.

# Acknowledgements

This report uses the public ImmunoMatch model interface and example-data workflow as a baseline for local engineering evaluation.

# References

[1] Guo, D., Dunn-Walters, D. K., Fraternali, F. et al. ImmunoMatch learns and predicts cognate pairing of heavy and light immunoglobulin chains. *Nature Methods* **23**, 106–117 (2026). https://doi.org/10.1038/s41592-025-02913-x.

[2] Wolf, T. et al. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 38–45 (2020).

[3] Lhoest, Q. et al. Datasets: A community library for natural language processing. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 175–184 (2021).